

# Demystifying a Legacy Teradata Notebook

Breaking a Monolithic Jupyter Notebook into Clean, Readable Cells

Your Name

Your Department / Course

September 11, 2026

# Outline

- 1 Introduction and Context
- 2 Environment and Access
- 3 The Monolithic Notebook
- 4 Cleanup Strategy
- 5 Teradata vs. Hive
- 6 Practical Workflow
- 7 Summary and Next Steps

# What This Project Is About

- We are given a **Jupyter Notebook** that begins with a **Teradata connection**.
- The notebook lives on a **Jupyter server**.
- The notebook is **not** a clean, modular notebook.
- It contains **one very large cell** with:
  - A large SQL query,
  - Python for loops that inject strings into the SQL,
  - Multiple smaller pandas DataFrames,
  - Business logic mixed with data access.
- Goal: **demystify, break down, and clean up** the notebook.

# Why This Matters

## The Problem

A single monolithic cell is hard to:

- Read,
- Debug,
- Explain,
- Reuse,
- Hand over to others (or to Copilot).

## The Solution

Split the code into smaller, logical cells and save intermediate outputs (e.g., DataFrame headers) so the workflow becomes transparent.

# What We Are *Not* Doing

- We are **not** rewriting the business logic.
- We are **not** changing the SQL semantics.
- We are **not** using Copilot for this phase.

## Why avoid Copilot for now?

We are preparing clean, structured input *for* Copilot.  
Using Copilot too early adds noise and confusion.

# Standard Procedure to Run the Notebook

1. Log in to the server using **PuTTY**.
2. Switch to the **Power Broker** account.
3. Navigate to our project folder.
4. Run the notebook command.
5. The notebook fetches the password via `config.py`.
6. Connect to **Teradata**.

## Note

Teradata is a legacy, classical data warehouse with its own computational framework. For this course, it is important to understand how it differs from **Hive**.

# Typical Access Flow

```
1      # Log in via PuTTY (outside the notebook)
2      # Then on the server:
3      sudo su - powerbroker
4      cd /path/to/our/folder
5      jupyter nbconvert --to notebook --execute our_notebook.ipynb
```

Listing 1: Shell commands (illustrative)

# What the Big Cell Looks Like

```
1  # config.py provides the password
2  from config import get_password
3
4  password = get_password()
5  conn = teradata.connect(..., password=password)
6
7  # One huge query with Python string injection
8  base_query = """
9  SELECT ...
10 FROM ...
11 WHERE month = '{month}'
12 """
13
14 for month in months:
15     query = base_query.format(month=month)
16     df = pd.read_sql(query, conn)
17     # ... lots of pandas logic ...
18     # ... more smaller DataFrames ...
19     # ... final output ...
```

# Why the Query Is So Big

- To **iteratively inject strings** into SQL, we need Python.
- A for loop (e.g., over months) injects a value into the query.
- Each iteration produces a separate query result.
- The original author wrote the loop and the query together.
- Over time, more and more logic was added into the same cell.

## Key Insight

The big query is not big because the SQL is complex.  
It is big because **Python and SQL are entangled**.

# What Else Is Inside the Big Cell?

- Multiple pandas DataFrames are created.
- Each DataFrame is separated into its own logic.
- Intermediate results are used to build the next step.
- The final output is saved or returned.

## Our Job

Separate the logic into different blocks and find out what is happening at each step.

# Step-by-Step Cleanup Plan

1. **Identify** the major logical sections in the big cell.
2. **Split** them into smaller Jupyter cells.
3. **Preserve** the original code (do not change logic).
4. **Save** intermediate outputs:
  - DataFrame headers,
  - Shapes,
  - Sample rows,
  - Column definitions.
5. **Document** what each cell does.

# Example: Splitting the Big Cell

## Before

- One cell
- 500+ lines
- SQL + Python + pandas
- Hard to debug

## After

- Cell 1: Imports and config
- Cell 2: Connection
- Cell 3: Query template
- Cell 4: Loop over months
- Cell 5: DataFrame 1
- Cell 6: DataFrame 2
- Cell 7: Final output

## Example: Cleaned Cell Structure

```
1  # Cell 1: Imports
2  import pandas as pd
3  from config import get_password
4
5  # Cell 2: Connection
6  password = get_password()
7  conn = teradata.connect(..., password=password)
8
9  # Cell 3: Query template
10 base_query = """
11 SELECT ...
12 FROM ...
13 WHERE month = '{month}'
14 """
15
16 # Cell 4: Loop over months
17 monthly_dfs = {}
18 for month in months:
19     query = base_query.format(month=month)
```

# Saving Intermediate Outputs

- For each DataFrame, save:
  - `df.head()`,
  - `df.shape`,
  - `df.columns.tolist()`,
  - `df.dtypes`.
- This makes it easy to:
  - Verify data,
  - Compare months,
  - Explain to others,
  - Prepare input for Copilot.

## Remember

We are **not** touching the code. We are only **reorganizing** it.

# Defining Columns at the Start

- At the top of the notebook, define the columns we care about.
- Example:

## Column definitions

```
KEY_COLUMNS = [  
    "customer_id",  
    "month",  
    "revenue",  
    "product_category",  
]  
  
DATE_COLUMNS = ["month", "transaction_date"]
```

- This makes the notebook self-documenting.
- It also helps when splitting the big cell.

# Teradata vs. Hive: Why It Matters

| Aspect   | Teradata               | Hive                        |
|----------|------------------------|-----------------------------|
| Type     | Classical RDBMS / MPP  | Hadoop-based data warehouse |
| Language | SQL (Teradata dialect) | HiveQL (SQL-like)           |
| Compute  | Proprietary engine     | MapReduce / Tez / Spark     |
| Storage  | Proprietary            | HDFS / cloud storage        |
| Use case | Legacy enterprise DW   | Big data analytics          |

## Takeaway

Understanding both helps you know *why* the code is written the way it is.

# Why We Still Use Teradata

- It is a **very old, classical, legacy** place to keep data.
- It has its own **computational framework**.
- Many enterprises still rely on it for critical reporting.
- The notebook we are cleaning is part of that legacy pipeline.

## Be Careful

Teradata SQL dialect may differ from Hive SQL.

Do not assume a query will run the same way on both.

# Recommended Workflow

1. Log in via PuTTY.
2. Switch to Power Broker account.
3. Navigate to the project folder.
4. Open the notebook.
5. Create a **copy** of the original notebook.
6. In the copy, split the big cell into smaller cells.
7. Run each cell and save intermediate outputs.
8. Add markdown explanations for each section.
9. Compare the final output with the original.

## Example: Saving Outputs to Disk

```
1  # After splitting, for each important DataFrame:
2  df.head(20).to_csv("output/df_head.csv", index=False)
3  with open("output/df_columns.txt", "w") as f:
4  f.write("\n".join(df.columns.tolist()))
5
6  # Also save a small summary
7  with open("output/df_summary.txt", "w") as f:
8  f.write(f"Shape: {df.shape}\n")
9  f.write(f"Columns: {df.columns.tolist()}\n")
10 f.write(f"Dtypes:\n{df.dtypes}\n")
```

Listing 4: Save DataFrame headers and samples

# What Good Looks Like

- A notebook with **clear sections**:
  - Imports,
  - Configuration,
  - Connection,
  - Query templates,
  - Loops,
  - DataFrame processing,
  - Final output.
- Each cell does **one thing**.
- Intermediate outputs are **saved and visible**.
- The original logic is **unchanged**.

# Summary

- We start with a monolithic Jupyter notebook connected to Teradata.
- The big cell mixes SQL, Python loops, and pandas logic.
- Our job is to **demystify** and **clean up** the notebook.
- We split the big cell into smaller, logical cells.
- We save intermediate outputs (headers, shapes, samples).
- We avoid Copilot for now to keep the analysis clean.
- We understand Teradata vs. Hive to better interpret the code.

# Next Steps

1. Get access to the actual notebook.
2. Identify the major logical blocks.
3. Create a cleaned copy.
4. Split the big cell step by step.
5. Save outputs for each DataFrame.
6. Document everything in markdown.
7. Only then consider using Copilot for further analysis.

## Remember

Clean code is not about rewriting.  
It is about making the existing code **understandable**.

Thank you!

Feel free to ask questions or share your notebook for a live walkthrough.